FINAL PROJECT PROPOSAL

Air Quality Planning for Outdoor Events

We propose a cloud application for a Chicago-area outdoor fitness operator who assigns recurring classes to venues in different neighborhoods and chooses among morning and afternoon start times. The operator uses historical patterns when setting the schedule and reviews recent local conditions when deciding whether to move a session to a backup venue or time.

The scheduling decision

Which candidate venues and recurring session windows have historically had lower PM2.5 concentrations and fewer sustained elevated episodes? We will compare seasonal distributions, pair morning and afternoon windows within the same monitor and date, and count consecutive elevated hours under a stated analytical threshold. Missing hours break episode runs. Neighborhood comparisons will show monitor proximity, sample counts and coverage gaps. The historical baseline will use AQS observations.

Data and cloud architecture

Nationwide EPA AirData. Pull every published hourly file from 1990 through the latest available year for all reporting monitors and criteria pollutants: ozone, SO2, CO, NO2, PM2.5 (FRM/FEM and non-FRM/FEM), PM10 and lead. Include all available hourly lead records and document empty files and pollutants measured on longer schedules. Keep the full national history upstream and derive the Chicago-area study tables after processing. Use the AQS site and monitor listings and follow the EPA PM2.5 data advisory. [1, 5]

Distributed processing. Land versioned ZIPs and a source manifest in Google Cloud Storage. Stage decompressed CSVs and run cloud PySpark across the full archive using explicit schemas. Normalize timestamps and units by pollutant, preserve monitor identifiers, methods and qualifiers, and reconcile duplicates and revisions. Write Parquet partitioned by year and pollutant. Cloud SQL holds normalized operational records and ingestion state. Snowflake serves curated facts, dimensions and analytical queries.

Live Chicago-area sensors. Poll the PurpleAir API every two minutes for public outdoor sensors in the metro, subject to a Week 1 key, quota and coverage check. Capture sensor ID, report time, arrival time, coordinates, A/B channel PM2.5 and humidity. Keep raw responses in Cloud Storage and incrementally process new reports by sensor ID and report time. Track staleness, duplicates, late arrivals and outages. [6, 7]

AirNow context. Continue collecting hourly AirNow observations and reconcile revisions to the preceding 72 hours. Streamlit displays preliminary AirNow AQI alongside separately labeled corrected PurpleAir PM2.5, coverage and freshness. Claude accesses curated Snowflake results through the MCP server. [2-4]

IMPLEMENTATION AND VALIDATION

Calibrate and match the live readings

Screen for missing humidity, stale reports and A/B channel disagreement. Aggregate quality-screened readings to UTC hours with a minimum completeness rule, then apply the documented EPA humidity-based PurpleAir correction appropriate to hourly averages. Match outdoor sensors to nearby eligible AQS reference or equivalent PM2.5 monitors using coordinates and a documented distance limit. Prefer collocations, record match distance and leave unsupported sensors unmatched. Join on overlapping UTC hours and use Chicago local time for scheduling displays. Compare raw and corrected bias and mean absolute error on held-out dates. Any local adjustment will be fit on separate overlapping reference data and evaluated on the holdout. Use preliminary AirNow monitor observations for live checks where eligible, then reconcile against AQS when available. [1-3, 7]

Team and milestones

Yujie Chen | Source access and cloud ingestion of historical and live data

Breanna Young | Normalized operational store and data quality reconciliation

Bravo Nie | Snowflake warehouse and exploratory and comparative analysis

Tongyan Liu | Streamlit application and Claude access through an MCP server

Weeks 1-2. Build the EPA file manifest, estimate rows and bytes from the catalog, test representative ZIPs and confirm the PurpleAir key, polling budget, required fields and metro coverage. Start collecting live readings.

Weeks 3-4. Launch the national backfill, validate PySpark partitions and warehouse queries, and establish sensor-to-monitor matches and calibration checks.

Weeks 5-6. Integrate the Chicago venue and session comparisons, AirNow context and live sensor view. Peer-review the pipeline and rehearse the integrated demo.

By Week 8. Complete the nationwide backfill and full PySpark run, with expected scale in the hundreds of millions of rows. Report actual rows and bytes, reconcile source counts by year and pollutant, and capture Spark runtime and worker configuration. Demonstrate successive live sensor updates, correction and matched-hour validation, and a Chicago operator comparing venues and session windows. Each owner presents their stage.

Sources

[1] EPA AirData downloads and monitor listings

[2] AirNow hourly file specifications

[3] AirNow data exchange guidelines

[4] AirNow public file archive

[5] EPA PM2.5 data advisory

[6] PurpleAir API overview

[7] EPA AirNow Fire and Smoke Map sensor methods